

Pertemuan 2

Linked List

Dosen:

Purbandini, S.Si., M.Kom.

Yutika Amelia Effendi, S.Kom., M.Kom.

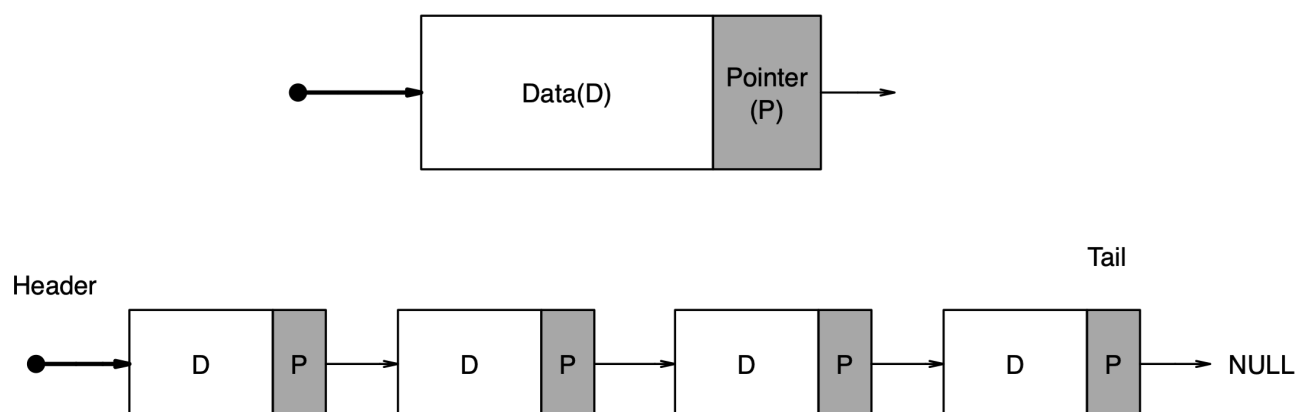
Rodik Wahyu Indrawan, S.ST., M.Tr.T.

A. Tujuan

1. Mahasiswa memahami konsep Linked List
2. Mahasiswa memahami Struktur dan Operasi(implementasi) Linked List

B. Dasar Teori

Secara konseptual, linked list merupakan deretan elemen yang berdampingan. Akan tetapi, karena elemen-elemen tersebut dialokasikan secara dinamis (menggunakan malloc). Single linked list atau biasa disebut linked list terdiri dari elemen-elemen individu, dimana masing-masing dihubungkan dengan pointer tunggal. Masing-masing elemen terdiri dari dua bagian, yaitu sebuah data dan sebuah pointer yang disebut dengan pointer *next*. Linked list dibentuk dengan cara menunjuk pointer *next* suatu elemen ke elemen yang mengikutinya. Elemen pada awal suatu list disebut *head*, dan elemen terakhir dari suatu list disebut *tail*. Pointer *next* pada elemen terakhir merupakan NULL, yang menunjukkan akhir dari suatu list.



Untuk mengakses elemen dalam linked list, dimulai dari head dan menggunakan pointer next dari elemen selanjutnya untuk berpindah dari elemen ke elemen berikutnya sampai elemen yang diminta dicapai. Ketika sebuah variabel dideklarasikan, terlebih

dahulu harus diinisialisasi. Demikian juga dengan pengalokasian penyimpan data secara dinamis. Node pada linked list merupakan suatu simpul(node) yang berisi pointer ke suatu data yang merupakan data dirinya sendiri. Model struktur dari linked list tersebut dalam C adalah sebagai berikut:

```
struct node {
    int data;
    struct node *next;
}
```

dilanjutkan dengan deklarasi dari pointer ke struktur di atas sebagai berikut:

```
struct node *head;
```

program untuk mengalokasikan sebuah node baru (Contoh `new`):

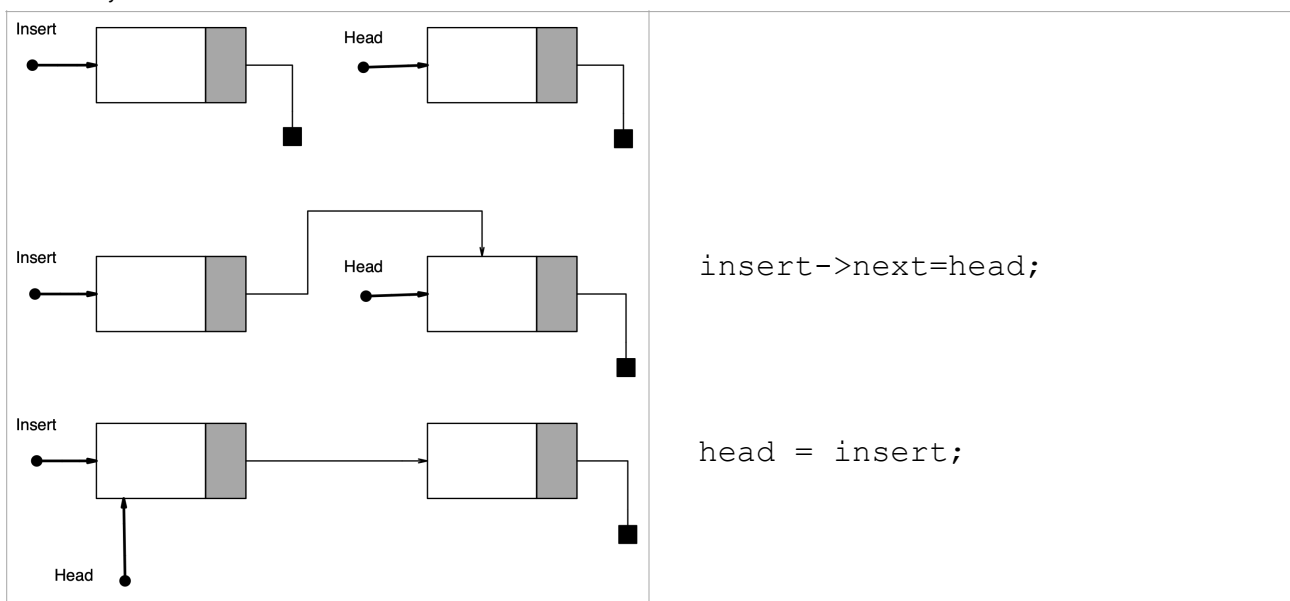
```
new = (list) malloc (sizeof(node));
new->data = data;
new->next=NULL;
```

B.1. Insert

Fungsi insert pada linked list meliputi :

- insert sebagai node awal (*head*) dari linked list, yaitu menambahkan sebuah node diawal list atau diawal node baru lainnya.

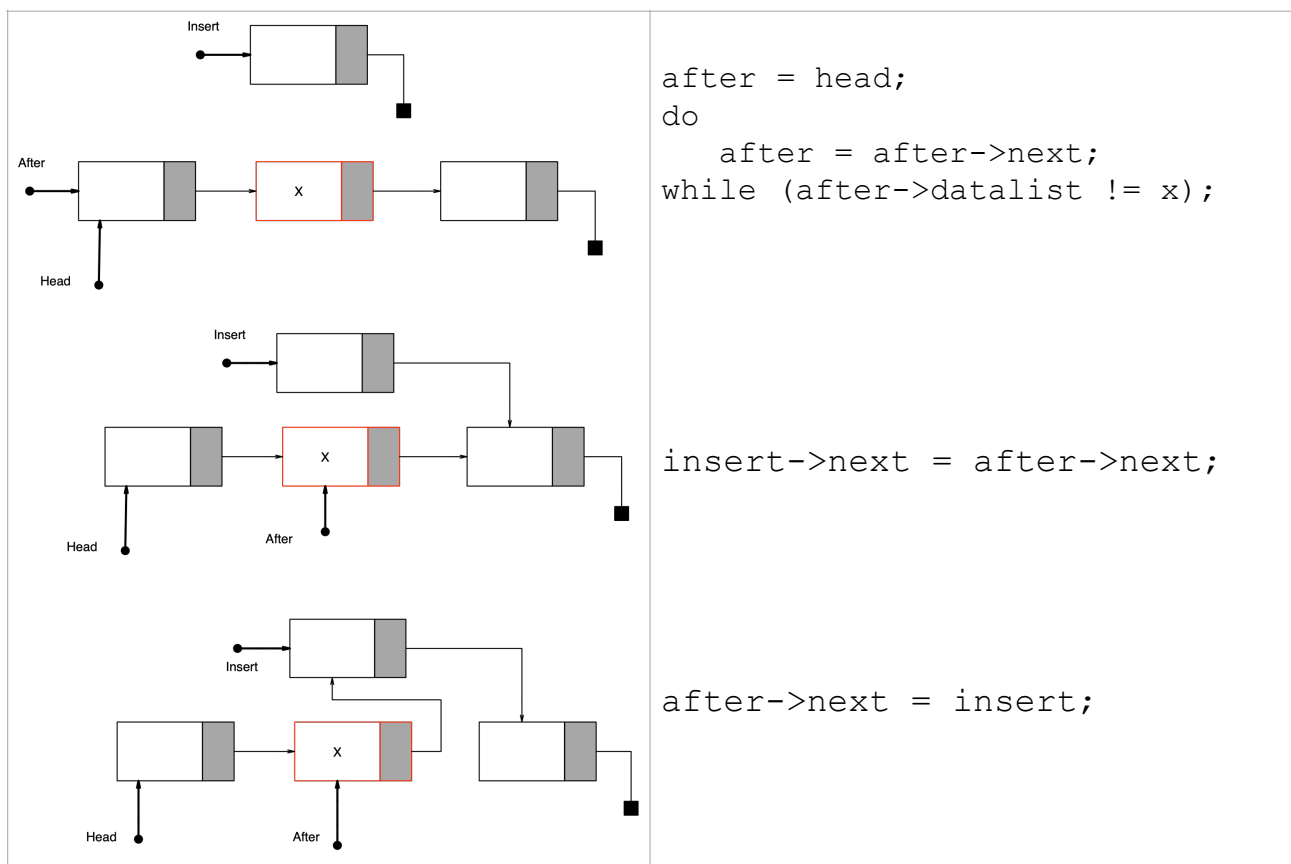
```
void insertashead(list insert)
{
    insert->next=head;
    head = insert;
}
```



- insert setelah node tertentu, yaitu menambahkan sebuah node setelah elemen tertentu yang terdapat pada list dengan cara pointer `next` dari elemen baru menunjuk elemen

setelah elemen tertentu (setelah elemen yang ditargetkan) pointer next elemen sebelumnya menunjuk ke elemen baru.

```
void insertafternode(int x, list insert)
{
    list after;
    after = head;
    do
        after = after->next;
    while (after->datalist != x);
    insert->next = after->next;
    after->next = insert;
}
```



- insert sebelum node tertentu, yaitu menambahkan suatu node sebelum element tertentu pada list dengan Langkah-langkah adalah telusuri list sampai elemen tertentu, catat juga elemen sebelumnya delanjutnya lakukan penyisipan sebelum elemen tertentu tersebut

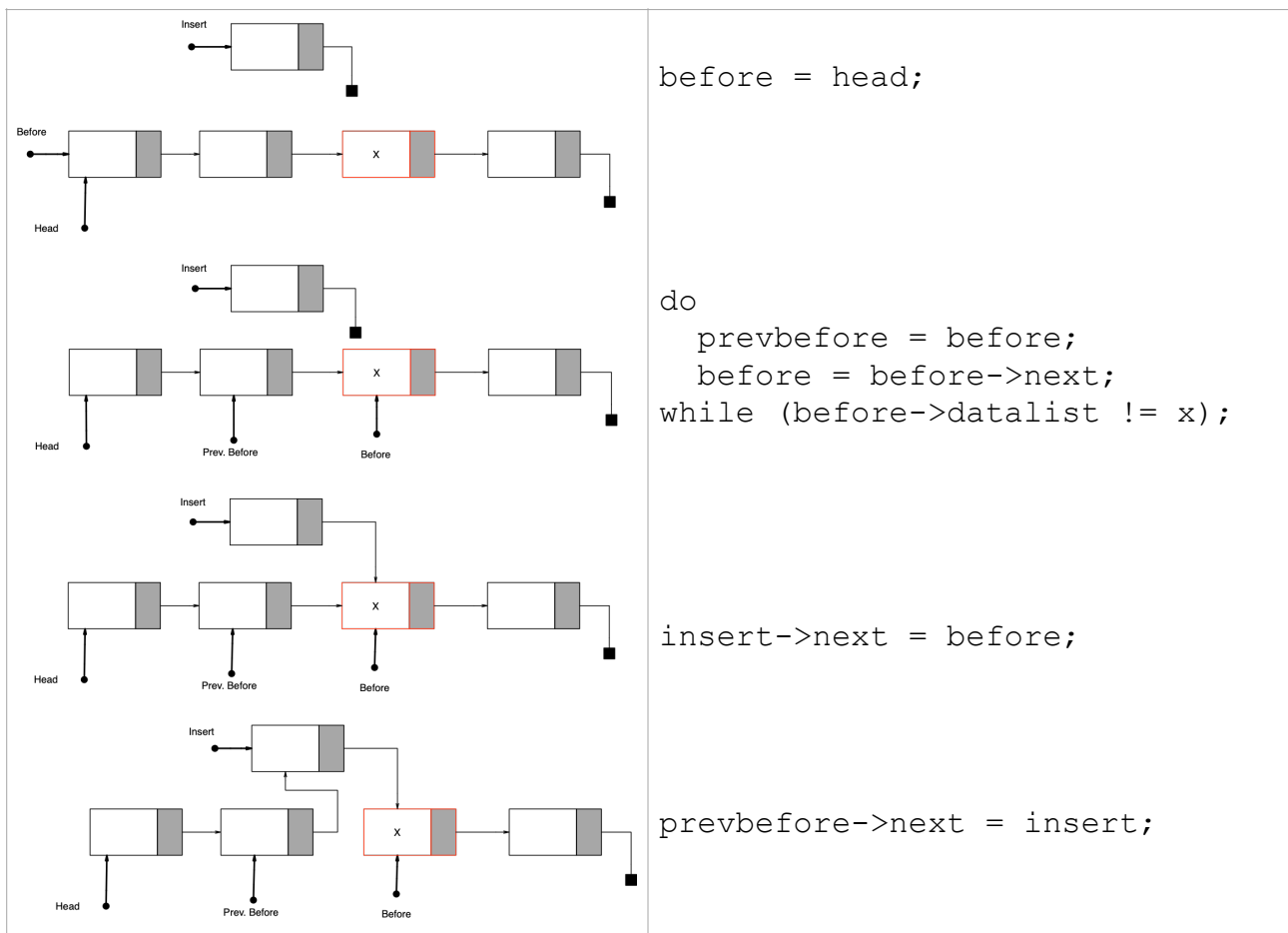
```
void insertbeforenode(int x, list insert)
{
    list before, prevbefore;
    if (head->datalist == x)
        insertashead(insert)
    else
```

```

{
    before = head;
    do
        prevbefore = before;
        before = before->next;
        while (before->datalist != x);

        insert->next = before;
        prevbefore->next = insert;
    }
}

```



- insert sebagai node akhir (*tail*) dari linked list, yaitu menambahkan suatu node di element terakhir pada suatu list, yaitu dengan cara telusuri list sampai elemen terakhir (`tail->next=NULL`) selanjutnya lakukan penambahan node setelah elemen terakhir

```

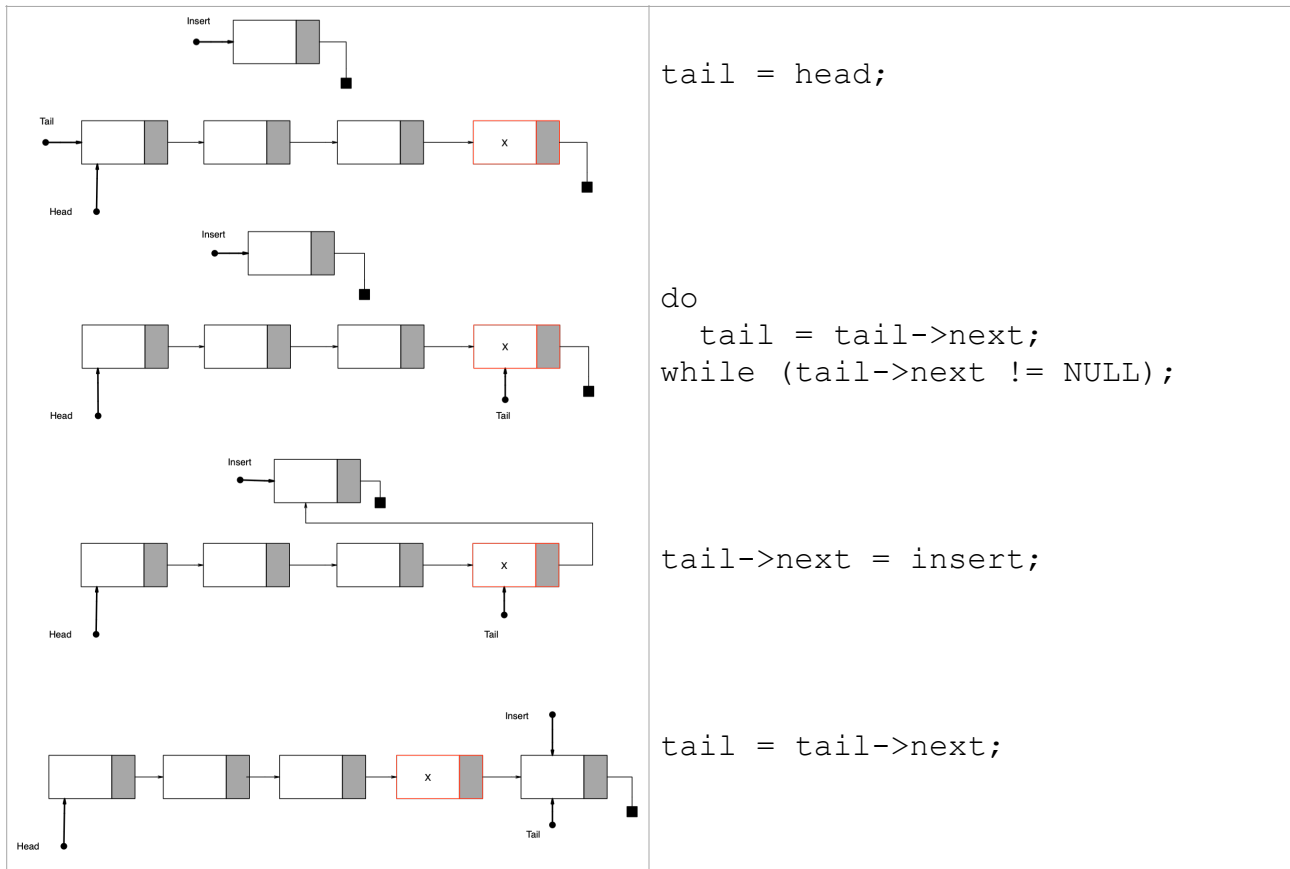
void insertastail(list insert)
{
    list tail;
    tail = head;
    do
        tail = tail->next;
    while (tail->next != NULL);
    tail->next = insert;
}

```

```

    }
    tail = tail->next;
}

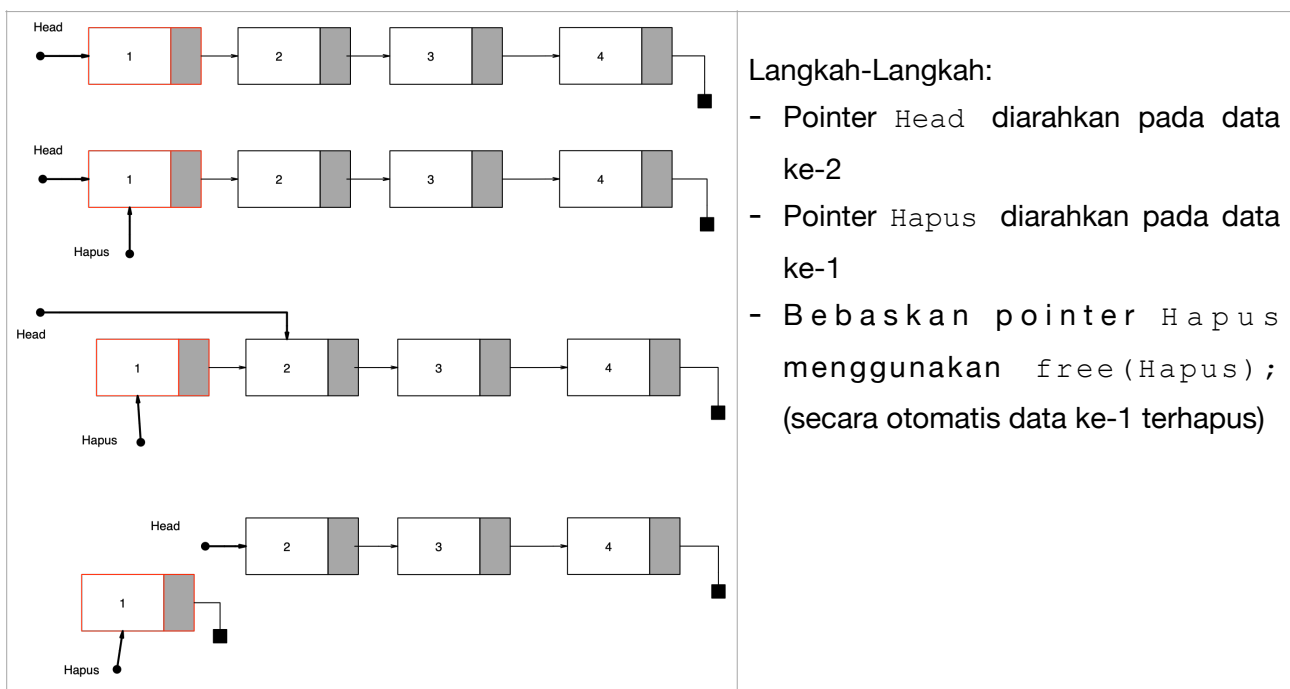
```



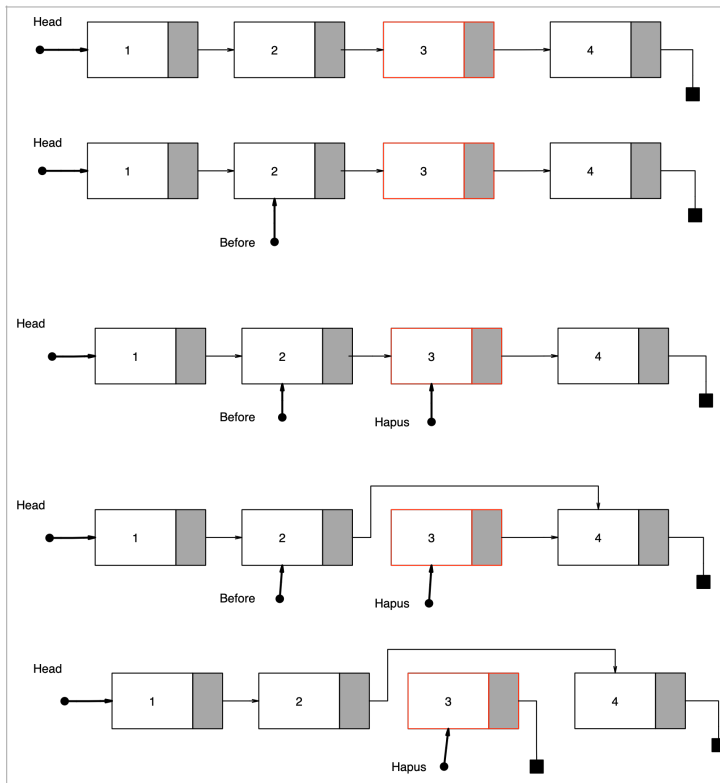
B.2. Delete

Fungsi *delete* node menggunakan `free(list)`; Operasi delete node pada linked list meliputi :

- *delete* sebagai simpul pertama(*head*) dari linked list



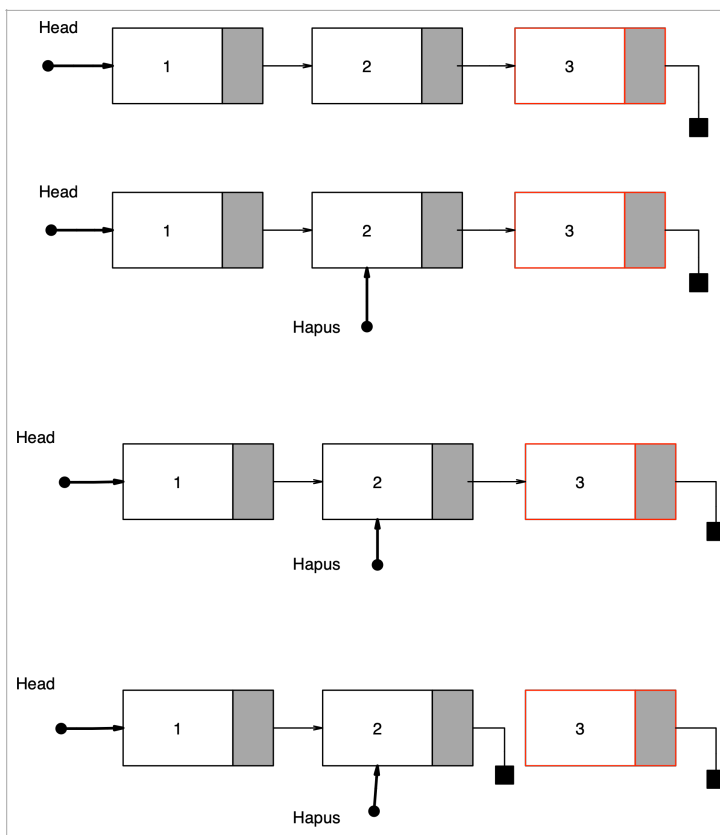
- *delete* setelah simpul tertentu



Langkah-Langkah:

- Arahkan pointer `before` s/d data yang ditunjuk
- Pointer `p` diarahkan pada `before->next`
- Arahkan pointer `before->next` pada `hapus->next`
- Bebaskan pointer `hapus` (secara otomatis data setelah simpul tertentu terhapus)

- *delete* simpul terakhir



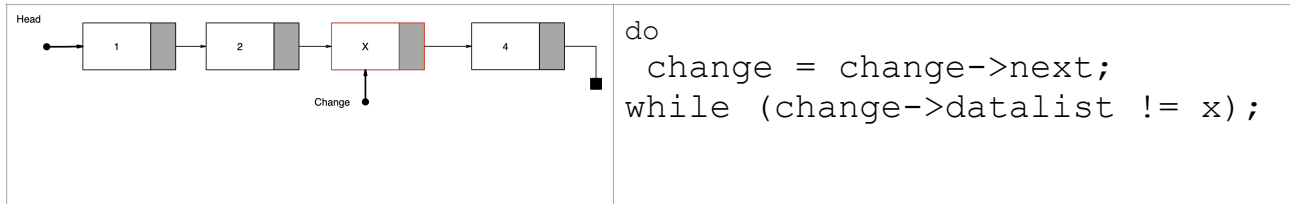
Langkah-Langkah:

- Telusuri simpul s/d `first->next = NULL`
- Arahkan pointer `hapus` ke `first`
- Bebaskan pointer `hapus->next` (Simpul Terakhir)
- Arahkan `hapus->next` ke `NULL`

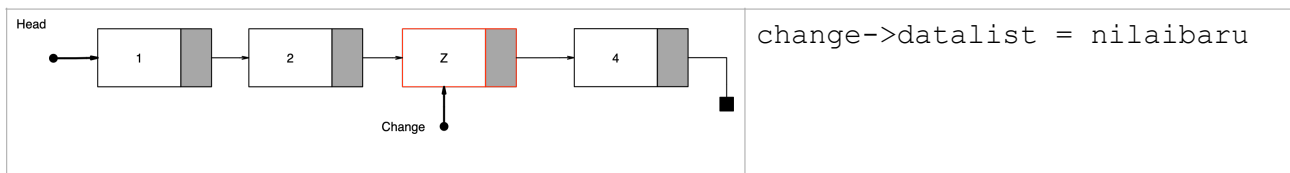
B.3. Merubah Isi data dari linked list

Untuk merubah isi dari suatu simpul pada linked list, maka yang harus dilakukan adalah:

- Menelusuri linked list sampai didapatkan simpul yang akan dirubah



- Merubah isi dari simpul yang sudah didapatkan



C. Kesimpulan

1. Linked list adalah sebuah struktur untuk menyimpan data yang bersifat dinamik
2. Beberapa operasi dapat diterapkan pada linked list seperti sisip(insert), hapus(delete)
3. Operasi-operasi yang ada pada linked list relatif lebih rumit jika dibandingkan dengan operasi-operasi pada struktur yang statis, karena dalam akses suatu linkedlist harus secara berurutan (sequencial)

D. Latihan

1. Buatlah program linked list untuk data mahasiswa (NIM dan Nilai), tambahkan prosedur untuk menambah dan menghapus data
2. Tuliskan sebuah program dengan menampilkan nilai rata-rata dari data Nilai yang dimasukkan pada linked list